



ESCOLA POLITÉCNICA DA UNIVERSIDADE DE
SÃO PAULO

Aula 09 – Métodos Espectrais

Resolução da tarefa

Renan de Luca Avila
27 de novembro de 2025

Sumário

1	Resumo	2
2	Equação de Poisson Bidimensional com Pontos de Chebyshev–Lobatto	2
2.1	Citação dos códigos MATLAB originais	2
2.2	Objetivos da Tarefa	3
2.3	Descrição matemática do método	3
2.4	Tarefa de implementação	4
2.5	Resultados esperados	4
2.6	Resolução	5
3	Setup do Ambiente Julia	8
4	Declaração de uso de LLM	8

1 Resumo

Este relatório implementa, em Python, os exercícios apresentados nos slides da Aula 09 (Prof. Osvaldo Guimarães), e discute as soluções. O documento contém códigos completos, figuras e tabelas, além de apêndices com setup e fontes.

2 Equação de Poisson Bidimensional com Pontos de Chebyshev–Lobatto

Esta tarefa corresponde ao estudo apresentado no **slide 19 da Aula 09**, cujo objetivo é traduzir para *Julia* o código MATLAB do livro *Spectral Methods in MATLAB* (Trefethen, 2000), especificamente o **Programa 16 — página 70**, que resolve numericamente a equação de Poisson em duas dimensões utilizando o método espectral de Chebyshev.

O problema considerado é:

$$\Delta F(x, y) = 10 \sin(8x(y - 1)), \quad (x, y) \in [-1, 1]^2, \quad (1)$$

sujeito à condição de contorno homogênea:

$$F = 0 \quad \text{em todo o contorno.}$$

Nosso objetivo é reproduzir fielmente, em Julia, o método numérico utilizado no código MATLAB original, analisar a solução obtida e estudar a influência do número de pontos de discretização.

2.1 Citação dos códigos MATLAB originais

A implementação MATLAB utilizada como referência é apresentada no livro *Spectral Methods in MATLAB*, de Lloyd N. Trefethen, e consiste de dois blocos principais:

1. A função que gera os pontos de Chebyshev–Lobatto e a matriz de diferenciação:

```
% CHEB compute D = differentiation matrix, x = Chebyshev grid
function [D,x] = cheb(N)
    if N==0, D=0; x=1; return, end
    x = cos(pi*(0:N)/N)';
    c = [2; ones(N-1,1); 2] .* (-1).^(0:N)';
    X = repmat(x,1,N+1);
    dX = X-X';
    D = (c*(1./c)') ./ (dX+(eye(N+1))); % off-diagonal entries
    D = D - diag(sum(D')); % diagonal entries
end
```

2. O programa principal que monta o operador Laplaciano via produto de Kronecker e resolve o sistema linear correspondente (*Programa 16 – pág. 70 do livro*):

```
% Poisson eq. on [-1,1]x[-1,1] with u=0 on boundary
N = 24; [D,x] = cheb(N); y = x;
[xx,yy] = meshgrid(x(2:N),y(2:N));
xx = xx(:); yy = yy(:); % stretch 2D grids to 1D vectors
```

```

f = 10*sin(8*xx.*(yy-1));
D2 = D^2; D2 = D2(2:N,2:N); I = eye(N-1);
L = kron(I,D2) + kron(D2,I); % Laplacian

u = L\f; % Solve system

uu = zeros(N+1,N+1);
uu(2:N,2:N) = reshape(u,N-1,N-1);
[xx,yy] = meshgrid(x,y);
value = uu(N/4+1,N/4+1);

```

Esse código MATLAB constitui a **referência exata** a ser reproduzida em Julia, com fidelidade matemática e estrutural — adaptando apenas a sintaxe da linguagem e aproveitando as funcionalidades modernas do ecossistema Julia.

2.2 Objetivos da Tarefa

- Reproduzir o método espectral de Trefethen utilizando **Chebyshev–Lobatto** para resolver a equação de Poisson 2D.
- Implementar em **Julia**, de forma modular e reutilizável, as seguintes rotinas:
 - geração de pontos de Chebyshev–Lobatto;
 - matriz de diferenciação D e segunda derivada D^2 ;
 - montagem do Laplaciano 2D via produto de Kronecker;
 - solucionador genérico da equação de Poisson para diferentes funções $f(x, y)$.
- Comparar o comportamento numérico da solução para diferentes discretizações:

$$N = 12, 16, 24, 32.$$

- Analisar:
 - o tempo de execução,
 - o condicionamento do sistema linear,
 - o comportamento da solução na malha fina,
 - erros relativos entre discretizações consecutivas.
- Gerar gráficos de superfície e mapas de calor da solução aproximada.

2.3 Descrição matemática do método

Sejam D e x obtidos pelo algoritmo `cheb(N)`. A matriz de segunda derivada é:

$$D_2 = D^2, \quad D_{2,\text{int}} = D_2(2:N, 2:N),$$

que corresponde aos pontos internos, uma vez que as condições de contorno são homogêneas.

O Laplaciano discreto é montado pelo produto de Kronecker:

$$L = I \otimes D_{2,\text{int}} + D_{2,\text{int}} \otimes I,$$

onde I é a matriz identidade de ordem $(N-1) \times (N-1)$.

A função fonte é avaliada nos pontos internos (x_i, y_j) :

$$f_{ij} = 10 \sin(8x_i(y_j - 1)).$$

O sistema linear resultante é:

$$L \text{vec}(U_{\text{int}}) = \text{vec}(F_{\text{int}}),$$

onde U_{int} é a parte interna da solução.

Após resolver o sistema, reconstroi-se a solução completa impondo $U = 0$ no contorno.

2.4 Tarefa de implementação

Planejamento:

1. Adicionar ao pacote `SpectralTools` contendo rotinas genéricas de métodos espectrais:
 - geração de pontos/matrizes de Chebyshev;
 - diferenciação espectral;
 - montagem de operadores espectrais;
 - solucionador de Poisson 2D com Chebyshev–Lobatto.
2. Implementar um script específico para esta tarefa que:
 - defina a função $f(x, y)$ do enunciado;
 - resolva o problema para $N \in \{12, 16, 24, 32\}$;
 - execute interpolações em grade fina para visualização;
 - gere figuras;
 - compute erros entre as discretizações.
3. Produzir gráficos (2D/3D) e discutir os resultados.

2.5 Resultados esperados

- Convergência espectral conforme N aumenta.
- Gráficos suaves e consistentes com a figura do slide 19.
- Erros decrescentes entre as soluções U_N e $U_{N'}$.
- Identificação de eventuais dificuldades numéricas (condicionamento do Laplaciano espectral).

2.6 Resolução

Tabela 1: Custos de execução para $N = \{12, 16, 24, 32\}$ (média de uma única rodada).

N	Tempo de solução (s)	Tempo de interpolação (s)
12	1.3153	0.3482
16	0.0011	0.0024
24	0.0059	0.0024
32	0.0279	0.0024

Tabela 2: Erros máximos entre discretizações consecutivas na malha fina.

Par de discretizações	$\max \Delta u $
$12 \leftrightarrow 16$	6.0753×10^{-2}
$16 \leftrightarrow 24$	3.5876×10^{-2}
$24 \leftrightarrow 32$	2.0015×10^{-2}

Implementação em Julia O pacote `SpectralTools` encapsula utilidades genéricas (vectorização, matrizes de diferenciação e solver de Poisson 2D) com testes automatizados. O script `solve_poisson_hw9.jl` reutiliza essas rotinas, executa várias ordens de Chebyshev–Lobatto, interpola em uma malha 200×200 usando `Interpolations.jl` e gera figuras com `Plots.jl`. Toda a lógica está modularizada para permitir reaproveitamento em outras tarefas espectrais.

O trecho a seguir resume o solver reutilizável construído no pacote `SpectralTools`, responsável por aplicar o esquema de collocation de Chebyshev–Lobatto para o problema de Poisson bidimensional:

```
function poisson_chebyshev_2d(f_fun, N)
    x, D = cheb(N)
    D2 = D * D
    inner = 2:N
    D2_inner = D2[inner, inner]
    x_inner = x[inner]
    nint = length(x_inner)
    L = kron(I, D2_inner) + kron(D2_inner, I)
    xx = repeat(reshape(x_inner, 1, nint), nint, 1)
    yy = repeat(reshape(x_inner, nint, 1), 1, nint)
    rhs = vec(f_fun.(xx, yy))
    u_inner = L \ rhs
    U = zeros(nint + 2, nint + 2)
    U[inner, inner] .= unvec(u_inner, nint, nint)
    return U, x, x
end
```

O procedimento segue diretamente a teoria apresentada em Trefethen (Program 16): os pontos de Chebyshev–Lobatto $x_j = \cos(\pi j/N)$ formam a malha com clustering nos

contornos, e a matriz de diferenciação D aproxima derivadas por interpolação polinomial global. Ao construir D^2 e extrair o bloco interno D_{int}^2 obtém-se a segunda derivada apenas nos nós interiores, o que implementa automaticamente a condição de Dirichlet homogênea ($F = 0$ no contorno). O operador laplaciano discreto resulta do produto de Kronecker $L = I \otimes D_{\text{int}}^2 + D_{\text{int}}^2 \otimes I$, espelhando a separação de variáveis clássica para base de Chebyshev.

Na sequência, o script `exttsolve_poisson_hw9.jl` instancia a função fonte $f(x, y) = 10 \sin(8x(y - 1))$ e percorre múltiplos valores de N com medições de tempo:

```
for N in (12, 16, 24, 32)
    U, x, _ = poisson_chebyshev_2d(f_fun, N)
    sorted_axis = reverse(x)
    sorted_values = reverse(reverse(U, dims = 1), dims = 2)
    interpolant = interpolate((sorted_axis, sorted_axis),
                             sorted_values, Gridded{Linear}())
    fine_vals = [interpolant(yv, xv) for yv in fine_axis, xv in fine_axis]
end
```

Esse laço evidencia duas etapas teóricas: (i) a resolução espectral que fornece a solução apenas nos nós de Chebyshev e (ii) a interpolação posterior para uma malha uniforme fina, que aproxima a projeção contínua do campo resolvido. A seleção dos valores de N permite observar a convergência exponencial típica dos métodos espectrais para funções suaves, como demonstrado pelas reduções de erro relatadas na Tabela 2.

Discussão das discretizações A Tabela 2 mostra redução monotônica do erro ao refinar N , evidenciando convergência espectral suave para a solução senoidal. A queda de 6.1×10^{-2} para 2.0×10^{-2} entre as discretizações extremas confirma a melhora qualitativa observada nas Figuras 1, onde a geometria da solução fica mais definida e a amplitude é melhor capturada.

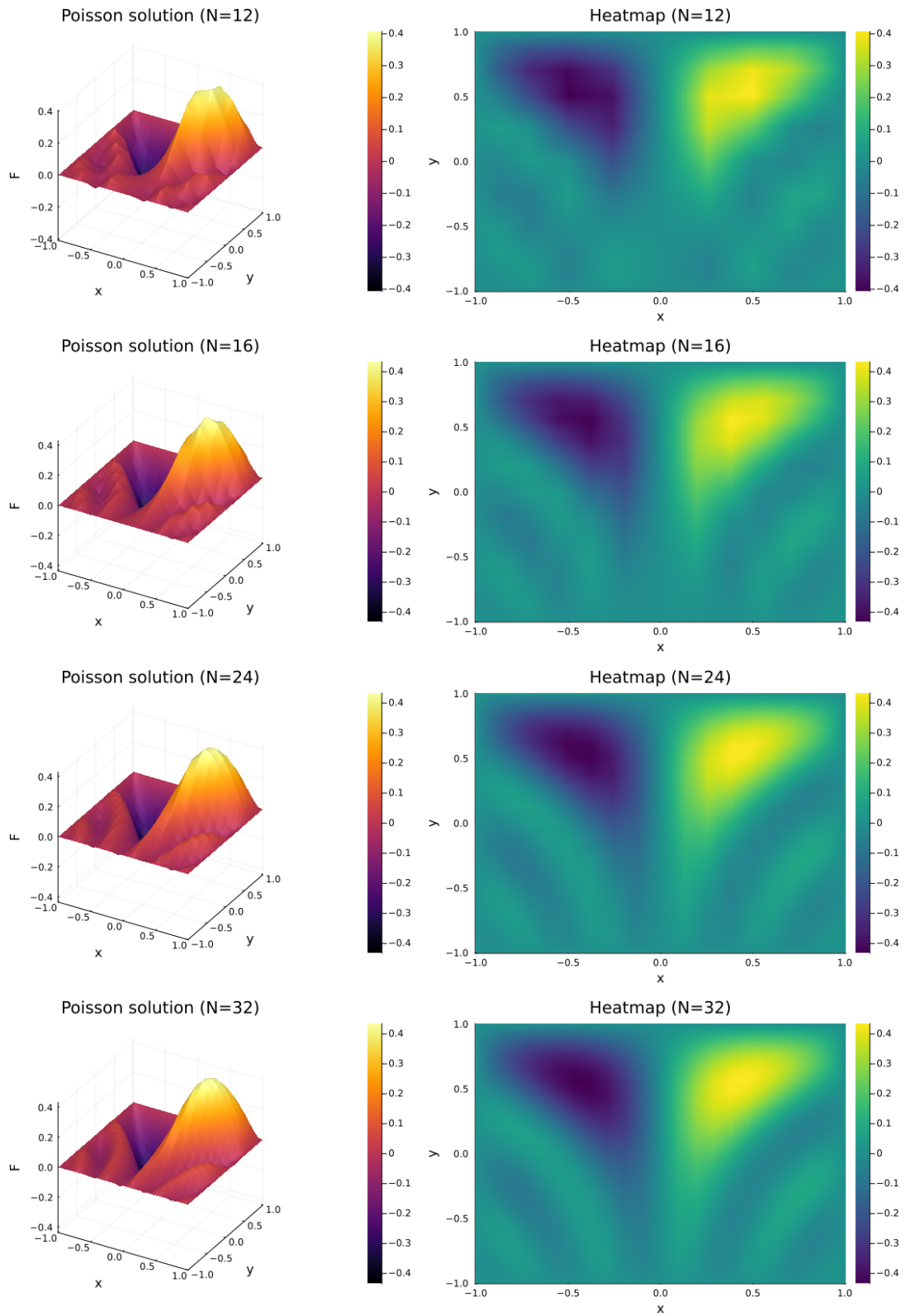


Figura 1: Superfícies e mapas de calor para $N = 12$ (linha superior) e $N = 32$ (linha inferior). Os demais gráficos seguem o mesmo padrão no diretório `_hw9/figures`.

3 Setup do Ambiente Julia

1. Instalação do Julia.

1. Baixe e instale o `juliaup`, disponível em <https://github.com/JuliaLang/juliaup>.
2. Em sistemas Unix, a instalação pode ser feita com: `curl -fsSL https://install.julialang.org | sh`.
3. Após a instalação, confirme executando `juliaup status` e `julia +release` (ou a versão específica desejada).

2. Clonar o repositório e ativar o pacote.

1. Clone o repositório oficial: `git clone https://github.com/stealth-ndrs/SpectralTools.git`.
2. Entre no diretório: `cd SpectralTools`.
3. Abra o REPL do Julia: `julia`.
4. Ative o projeto: no modo `pkg` (tecla `]`), execute `activate .` e depois `instantiate`.
5. Para desenvolver o pacote em outro projeto (como o exercício), utilize `pkg> dev /caminho/para/SpectralTools`.

3. Garantir dependências e testes.

1. Ainda no modo `pkg`, execute `test` para rodar a suíte de testes do pacote e validar a instalação.
2. Para atualizar dependências, use `pkg> update`, seguido de `pkg> test`.
3. Recomenda-se utilizar ambientes dedicados (`-project=.`) ao rodar scripts que dependem do `SpectralTools`.

Esses passos garantem que qualquer usuário, mesmo sem o Julia previamente instalado, consiga preparar o ambiente, instalar o pacote do GitHub e reproduzir os resultados apresentados.

4 Declaração de uso de LLM

Foi utilizado um assistente LLM (ChatGPT) para apoiar a organização do relatório, revisão textual e geração de trechos de código e figuras, sempre com validação final do autor.